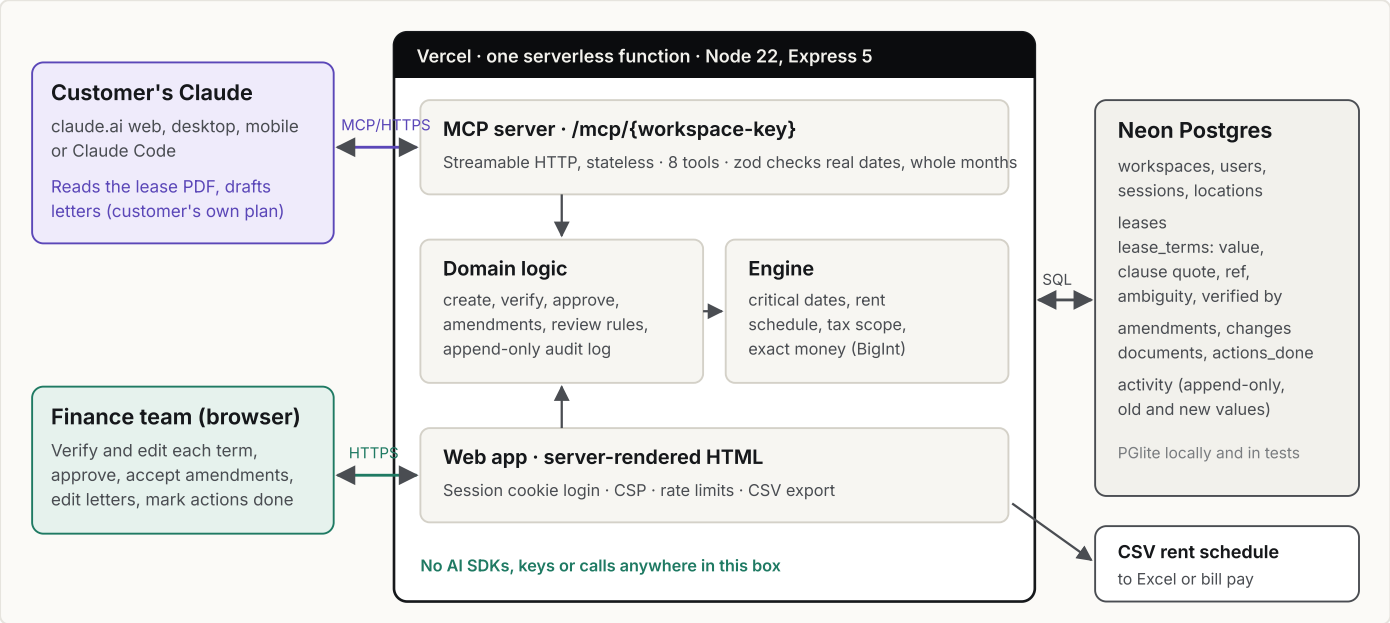
README · ARCHITECTURE

The customer's Claude does the reading. LeaseIntel stores, checks and calculates. No AI runs inside the app.

Part 1 shows what is **built and live today** at leaseintel.vercel.app. Part 2 shows the **proposed production system**, including how it would connect to Volopay. Anything not built is drawn with dashed lines and labelled "proposed".

01 Built today (live)



02 Data flow, step by step

1. **Request in Claude.** A finance lead attaches a lease PDF and says "Add this lease to LeaseIntel". Claude reads it on the customer's own plan.
2. **Claude calls `create_lease`** with up to 23 terms. Each term carries the exact clause quote, its reference and, if unclear, an ambiguity note. The server's instructions tell Claude to leave out what the lease does not state.
3. **The server checks the input** (real calendar dates, whole months, money to 2 decimals) and saves the lease, its location, every term as *unverified*, and an activity row, all in one database transaction. Claude gets back a link and the list of key terms still to verify.
4. **A person reviews in the browser.** Each term sits next to its quote. Saving a value marks it verified by that person, with old and new value logged. Approve stays locked until every key term is verified.
5. **Dates and rent are calculated on every read** from the stored terms (lock-in, exit notice, renewal deadline, escalations, expiry, monthly schedule), so they can never drift from the verified data.
6. **Later, Claude reads back** with `list_portfolio` / `get_lease`, sees which values a person corrected, and saves letters with `save_document` as drafts. A person edits and marks them sent.
7. **Amendments** arrive through `propose_amendment`. Old terms stay until a person accepts or rejects each change. If Claude changes a key term on an approved lease, it goes back to review.
8. **Finance exports** the month-by-month rent schedule as CSV.

03 Tools used

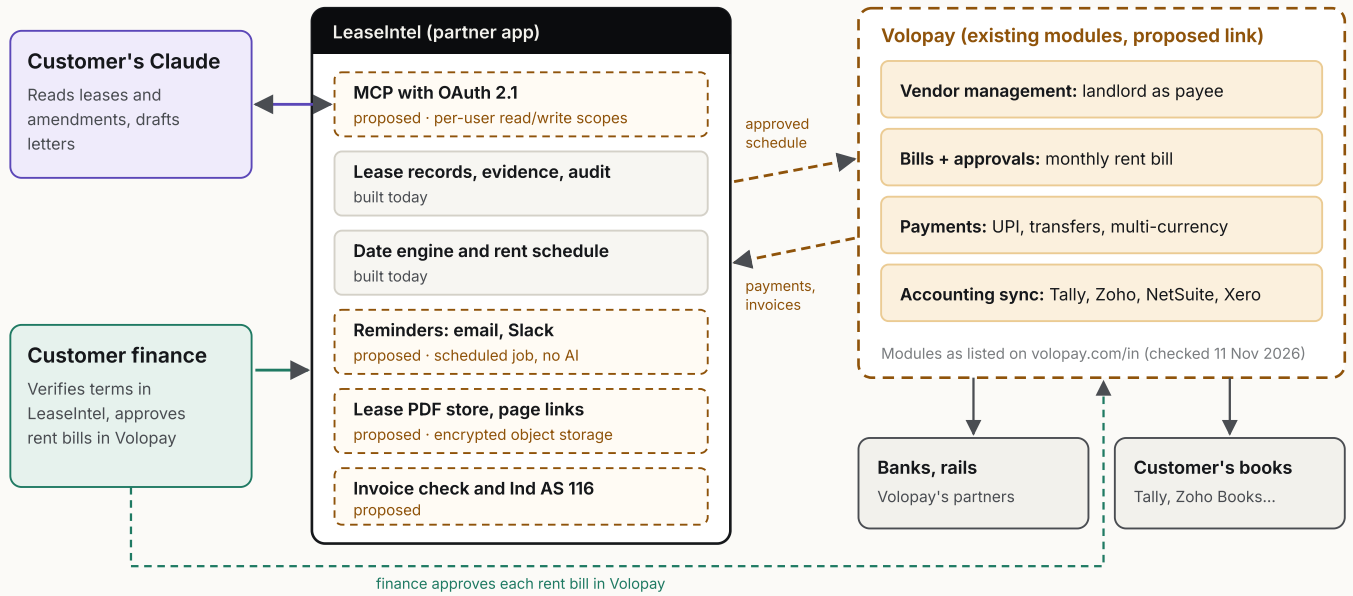
App	Node.js 22, Express 5, server-rendered HTML, vanilla JS, self-hosted fonts
MCP	@modelcontextprotocol/sdk 1.32, zod 3
Data	Neon Postgres via pg 8; PGlite for local runs and tests
Hosting	Vercel (deploys on push to GitHub main)
Quality	node:test, 57 tests incl. end-to-end through a real MCP client; pdftotext checks every quoted clause; headless Chrome for screenshots
Building	Claude as coding help (allowed by the brief), GitHub
AI inside	None. No AI SDK or key in the code or environment

Links. App `leaseintel.vercel.app` · login `demo@leaseintel.app` / `leaseintel-demo` · MCP `leaseintel.vercel.app/mcp/li_demo_volopay_2026` · code `github.com/pmInTheLoop/leaseintel`

PROPOSED PRODUCTION SYSTEM

LeaseIntel keeps the verified lease record. Volopay moves the money.

04 Proposed production architecture



05 Volopay versus partner responsibilities (proposed)

AREA	VOLOPAY	LEASEINTEL (PARTNER APP)	OTHER PARTNERS AND THE CUSTOMER
Reading leases, drafting letters	Nothing	Stores what Claude saves through MCP; never calls an AI itself	Customer's Claude (Anthropic), on the customer's own plan
Lease record and verification	Shows the lease reference on each rent bill	System of record: 23 terms, clause evidence, who verified what, full audit log	Customer's finance team verifies and approves
Deadlines and rent maths	Shows when a bill amount is due to change	Date engine, escalations, rent + CAM + tax schedule, reminders (proposed)	None
Landlord as vendor	Vendor record, bank details, vendor checks	Sends landlord name, location and lease ID	None
Paying rent	Recurring bill, approval flow, payment run, UPI, transfers, FX	Never moves money; sends the approved amount per month	Volopay's banking and payment partners
Invoice check	Flags a mismatch at bill approval	Expected amount for that month from the verified schedule	Landlord sends the invoice
Accounting	Sync to the customer's books (Tally, Zoho Books, NetSuite, Xero)	Ind AS 116 / IFRS 16 schedules (next step)	Customer's CA firm for accounting judgement
Access and security	Finance users, roles, SSO on the Volopay side	OAuth for MCP, workspace isolation, audit log	Anthropic handles the Claude side of OAuth
Hosting and data	Volopay's own platform	Vercel and Neon today; a security review before real data	Vercel, Neon

Why split it this way: Volopay already owns the payee, the approval and the payment, which are regulated and hard to build. LeaseIntel owns the contract knowledge that decides *what* should be paid and *when* it changes. Today the bridge is the CSV export; Volopay's site also lists CSV and SFTP imports, which could carry it before any API work.

BUILT VERSUS PROPOSED · ASSUMPTIONS · LIMITS · NEXT STEPS

What exists today, what production needs, and what I assumed.

06 What I built versus the proposed production system

PART	BUILT AND LIVE TODAY	PROPOSED FOR PRODUCTION
Claude access	MCP over HTTPS with a secret URL per workspace (can be rotated)	OAuth 2.1 with per-user tokens and read/write scopes
App access	Email and password, HttpOnly session cookie, login and signup rate limits, CSP	SSO, roles (viewer, verifier, approver, admin), rate limits in a shared store
Lease data	23 terms with clause quote, reference, ambiguity note, verifier and time; amendments; letters; append-only history	Same, plus the lease PDF stored with links from each term to its page
Calculations	Lock-in, exit notice, renewal deadline, escalations, expiry; monthly rent + CAM + tax; exact money; per-currency totals	Ind AS 116 / IFRS 16 schedules; business-day and holiday rules where a lease asks for them
Alerts	90-day "Needs your attention" list in the app; Claude can be asked any time	Email and Slack reminders at 90, 30 and 7 days; calendar feed
Payments	CSV export of the rent schedule	Volopay recurring rent bills, payment status and landlord invoices coming back
Operations	Vercel deploys from GitHub, Neon defaults, health endpoint, 57 automated tests	Backup and restore policy, monitoring and alerts, penetration test, load tests

07 Assumptions

- The customer has a Claude plan that allows a custom connector (the free plan allows one).
- Leases are PDFs Claude can read. Scanned pages depend on how well Claude reads them; a person verifies every key term either way.
- "N months before expiry" is counted back from the last day of the term, the safer reading. Calendar dates, no holiday rules.
- Tax is charged only where the agreement says it applies. If it is unclear, it is not charged and totals are marked incomplete.
- Price (₹399 per location per month) and the 1-year and 3-year targets are untested. No customer conversations yet.
- Volopay would expose vendor and bill APIs, or accept CSV/SFTP, for the link. Not verified.

08 Limitations today

- The MCP link is a shared secret; anyone with it can read and change that workspace. Fine for a demo, not for real data.
- The demo workspace is shared by all reviewers, so data can change; "Reset demo workspace" restores it.
- No PDF storage, so a reviewer checks Claude's quote against their own copy of the lease.
- No reminders are sent; LeaseIntel never sends emails or letters.
- Rate limits live in memory per serverless instance.
- Exercising a renewal option does not extend the term by itself; a person edits the expiry date.
- Totals are per currency; there is no FX conversion, by design.

09 Next steps

1. **Talk to 10 finance leads** at 10 to 60 location chains to test the problem, the price and who signs.
2. **OAuth for MCP and user roles** before any real lease data.
3. **Reminders** by email and Slack, plus a calendar feed.
4. **Store the PDF** and link each term to its page.
5. **Volopay pilot via CSV/SFTP:** landlord payees and recurring rent bills from the approved schedule, then an API link and invoice check.
6. **Ind AS 116 schedules** for the CA firms who audit these chains.

How to try it. Open `leaseintel.vercel.app` and log in with `demo@leaseintel.app` / `leaseintel-demo`. In Claude: Settings → Connectors → Add custom connector, URL `https://leaseintel.vercel.app/mcp/li_demo_volopay_2026`, OAuth empty. Attach a sample PDF from `leaseintel.vercel.app/samples` and say "Add this lease to LeaseIntel." The lease appears as Needs review; edit a term, approve, then ask Claude "What needs attention in the next 90 days?"